NATURAL LANGUAGE PROCESSING FOR AI

Problem Set 04

From Counting Words to Teaching Machines to Read

A friendly introduction to n-grams, similarity, neural language models, attention, and how AI models read and write

Wongel Dainel • **Mikiyas Shegaw**

The big idea

A language model gives every possible next word a probability. At first we get those probabilities by **counting**. Later we let a **neural network** learn them. In both cases we judge the model with the same ruler: how surprised was it by the real text? This problem set walks along that road, from counting words to attention.

Name: _____ **Date:** _____

Suggested time: 90–120 minutes **Total:** 100 marks

What you will learn

By the end of this problem set, you should be able to:

- turn raw text into clean tokens and simple count-based features;
- compute n-gram probabilities and score sentences with perplexity;
- measure how similar two texts are with cosine similarity and embeddings;
- see how a neural network can replace a table of counts (softmax, loss, gradients);
- explain how embeddings and attention let a model use context;
- read graphs of loss, temperature and attention weights;
- describe how GPT-style models choose words and how a voice assistant fits together.

A small guide to the symbols

You do not need to memorize these symbols immediately. Use this box whenever you need a reminder.

$P(b \mid a)$	probability that the next word is b, given that the word before it is a
$\text{count}(a\,b)$	how many times the pair $a\,b$ appears in the training text
`<s>`, `</s>`	markers for the start and the end of a sentence
V	vocabulary size: how many different words the model knows
$\mathbf{x} \cdot \mathbf{y}$	dot product: multiply matching entries, then add them up
$\cos\theta$	cosine similarity, $\dfrac{\mathbf{x} \cdot \mathbf{y}}{\lVert\mathbf{x}\rVert\,\lVert\mathbf{y}\rVert}$
$\mathbf{z}$	logits: the raw scores a network produces before softmax
$p_i = \dfrac{e^{z_i}}{\sum_j e^{z_j}}$	softmax: turns scores into probabilities that add up to 1
$L = -\ln p$	loss (negative log-likelihood): small when the true word gets a high probability
PP	perplexity, $\exp(\text{average loss})$: lower is better

How to work

- Show your working. Marks are given for the method, not only for the final number.
- $\ln$ means the natural logarithm. Round to 3 decimal places unless the question says otherwise.
- The parts get harder as you go: ★ warm-up, ★★ core, ★★★ stretch, ★★★★ challenge.
- For short written answers, write 2 to 4 sentences and give a reason.
- When a question uses a graph, read it carefully and state the values you read from it.

Part	Topic	Level	Marks
A	Preparing text	★	12
B	N-grams and probabilities	★	16
C	Judging a model: perplexity	★★	10
D	Similarity and meaning	★★	14
E	From counts to a neural network	★★★	18
F	Embeddings and the neural language model	★★★	10
G	How GPT chooses words, and speech	★★★	8
H	Attention	★★★★	12
	Total		**100**

Part A: Preparing text

★ · Warm-up · 12 marks

Computers cannot use raw text directly. Figure 1 shows the usual steps that clean it first.

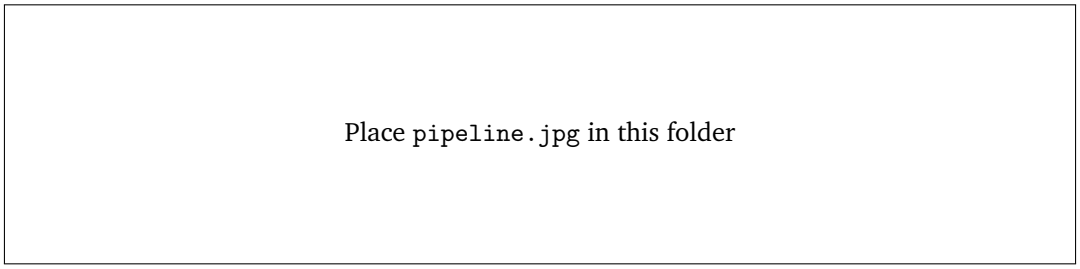


Figure 1: A typical text-preparation pipeline: raw text, tokens, cleaned tokens, numbers.

Question 1 [3 marks]

A search tool must treat “studies”, “studying” and “studied” as the same idea.

(a) In one sentence each, explain what *stemming* and *lemmatisation* do. [2]

(b) Which one would turn “better” into “good”? Why? [1]

Question 2 [3 marks]

A review tool removes stop words before it measures sentiment. Its stop-word list is {the, was, not, at, all}. The review is: “The movie was not bad at all.”

(a) Write the review after the stop words are removed. [1]

(b) Why is this a problem for a sentiment tool? [1]

(c) Give one fix. [1]

Question 3 [3 marks]

Two headlines: “dog bites man” and “man bites dog”.

(a) Write the bag-of-words vector of each headline, using the vocabulary [dog, bites, man]. [1]

(b) What information does a bag of words throw away? [1]

(c) List the bigrams of each headline. Do they tell the headlines apart? [1]

Question 4 [3 marks]

A corpus has $N = 8$ articles. Article X has 40 words. The word “the” appears 6 times in X and appears in all 8 articles. The word “vaccine” appears 4 times in X and appears in only 2 articles. Use $\text{tf} = \dfrac{\text{count in the article}}{\text{article length}}$ and $\text{idf} = \ln \dfrac{N}{\text{number of articles containing the word}}$.

(a) Compute tf $\times$ idf for “the” and for “vaccine”. [2]

(b) Which word describes article X better? Explain in one sentence. [1]

Part B: N-grams and probabilities

★ · Core · 16 marks

Use this tiny training text for Questions 5 to 10. Each line is a separate sentence, with start and end markers added. **Do not count across sentences.**

```
<s> i like pizza </s>
<s> i like pasta </s>
<s> i eat pizza </s>
<s> you like pizza </s>
<s> i like pizza </s>
```

Use $P(w_2 \mid w_1) = \dfrac{\text{count}(w_1\, w_2)}{\text{count}(w_1)}$. As a check, the text contains 20 bigrams in total. Figure 2 shows how often each bigram occurs.

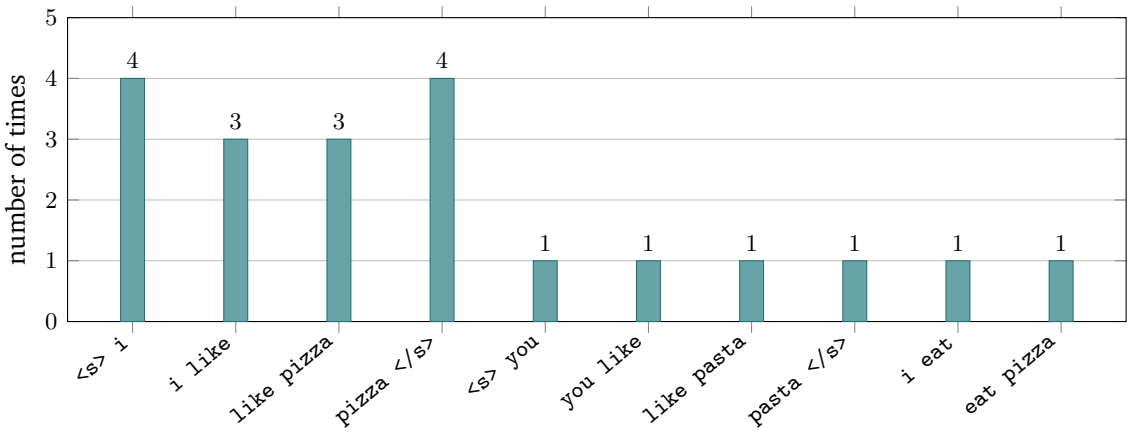


Figure 2: Bigram counts in the training text of Part B.

Question 5 [2 marks]

Use Figure 2 (and add bars where needed) to fill in the four counts (half a mark each):

count(i like) =_____ count(like pizza) =_____ count(pizza </s>) =_____ count(like) =_____

Question 6 [3 marks]

Compute (a) $P(\text{pizza} \mid \text{like})$, (b) $P(\text{pasta} \mid \text{like})$, (c) $P(\text{eat} \mid \text{i})$. Then check that (a) + (b) $= 1$ and explain why this must happen here. [3]

Question 7 [3 marks]

The probability of a sentence is the product of its bigram probabilities. Compute the probability of "i like pasta":

$$P(\text{i} \mid \text{<s>}) \times P(\text{like} \mid \text{i}) \times P(\text{pasta} \mid \text{like}) \times P(\text{</s>} \mid \text{pasta}).$$

Write each factor and the final product. [3]

Question 8 [3 marks]

Now look at the sentence "you eat pizza".

(a) What is $P(\text{eat} \mid \text{you})$? [1]

(b) What probability does the model give the whole sentence? [1]

(c) Why is this a problem, since the sentence is perfectly sensible? [1]

Question 9 [3 marks]

Add-one smoothing gives every pair a pretend extra count of 1:

$$P_{\text{add1}}(w_2 \mid w_1) = \frac{\text{count}(w_1\, w_2) + 1}{\text{count}(w_1) + V}.$$

The model's vocabulary is {i, you, like, eat, pizza, pasta, </s>}, so $V = 7$.

(a) Compute $P_{\text{add1}}(\text{eat} \mid \text{you})$. [1]

(b) Compute $P_{\text{add1}}(\text{pizza} \mid \text{like})$ and compare it with the unsmoothed $3/4$. [1]

(c) What does smoothing do to the probability of events that were seen? [1]

Question 10 [2 marks]

(a) A sentence has n words. How many trigrams does it contain? Check with "the cat sat on the mat". [1]

(b) How many 5-grams are in a text of 20 words? [1]

Part C: Judging a model: perplexity

★★ · Core · 10 marks

How do we say that one language model is better than another? We ask how *surprised* it is by real text. For a test text with N predicted tokens (including </s>):

$$\mathrm{PP} = P(\text{text})^{-1/N} = \exp\Big(\frac{1}{N}\sum_{t} -\ln p_t\Big).$$

Lower perplexity means the model is less surprised, so it is better. Use the unsmoothed model from Part B.

Question 11 [3 marks]

Compute the perplexity of "i like pasta". Here $N = 4$ (the predictions are for i, like, pasta and </s>). Use your answer from Question 7. [3]

Question 12 [3 marks]

(a) A silly model ignores the text and picks uniformly among all $V = 7$ words at every step. Show that its perplexity is exactly 7 for any text length N, and say what this tells you about how to read a perplexity of 1.607. [2]

(b) Figure 3 shows the perplexity of three real models on the same large news test text. Which model is best, and what does the graph tell you about adding more context? [1]

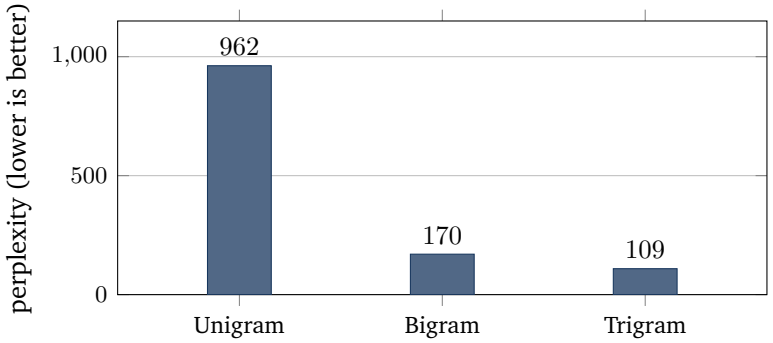


Figure 3: Perplexity of unigram, bigram and trigram models on a news test set (values from Jurafsky & Martin, *Speech and Language Processing*, chapter on n-gram language models).

Question 13 [4 marks]

Now compute the perplexity of “i like pizza”.

(a) Find the probability of the sentence. [1]

(b) Find its perplexity. [2]

(c) The perplexity is lower than for “i like pasta”. Explain why in terms of the training text. [1]

Part D: Similarity and meaning

★★ · Core · 14 marks

Question 14 [3 marks]

Use the vocabulary [i, like, pizza, pasta, eat]. The three documents are

A: "i like pizza" $= (1, 1, 1, 0, 0)$, B: "i like pasta" $= (1, 1, 0, 1, 0)$, C: "i eat pizza" $= (1, 0, 1, 0, 1)$.

Compute $\cos(A, B)$, $\cos(A, C)$ and $\cos(B, C)$. Which pair is the least similar? [3]

Question 15 [3 marks]

You compare a long report with its short summary. Their word-count vectors are $\mathbf{u} = (10, 20, 30)$ for the report and $\mathbf{v} = (1, 2, 3)$ for the summary.

(a) Compute the straight-line (Euclidean) distance between them. [1]

(b) Compute their cosine similarity. [1]

(c) Which measure is better for deciding that they are about the same topic? Why? [1]

Question 16 [4 marks]

In a toy 2-dimensional embedding space: doctor $= (0.9, 0.8)$, nurse $= (0.8, 0.9)$, banana $= (0.1, -0.9)$. Figure 4 draws the three vectors.

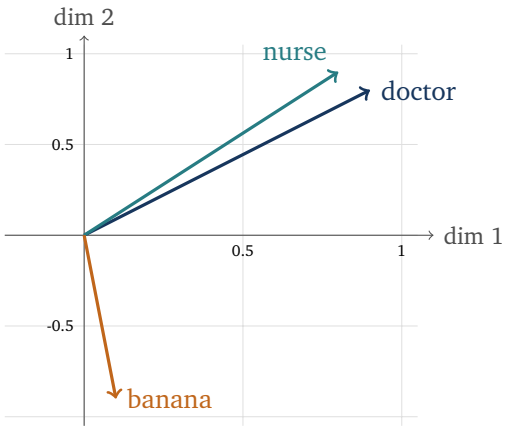


Figure 4: Three word vectors in a toy 2-dimensional embedding space.

(a) Compute $\cos(\text{doctor}, \text{nurse})$. [2]

(b) Compute $\cos(\text{doctor}, \text{banana})$. [1]

(c) What does the negative sign mean for the word pair in (b)? Link your answer to the picture. [1]

Question 17 [4 marks]

A help centre wants the customer message “my payment didn’t go through” to find the article “Troubleshooting failed transactions”. The two texts share no words. Explain why word counting fails and embeddings work, and name one thing that can still go wrong with embedding similarity. [4]

Part E: From counts to a neural network

★★★ · Stretch · 18 marks

In Part B we stored probabilities in a table of counts. A neat trick, used in many tutorials on character-level and word-level language models, is to replace the table with a tiny **neural network** that *learns* the same probabilities. The recipe has five steps:

1. turn the current word into a one-hot vector $\mathbf{x}$;
2. multiply by a weight matrix W to get scores (logits) $\mathbf{z} = \mathbf{x}W$;
3. apply softmax to turn the scores into probabilities $\mathbf{p}$;
4. compute the loss $L = -\ln p_{\text{true}}$, where p_{true} is the probability given to the real next word;
5. nudge W a little in the direction that lowers L (gradient descent), then repeat.

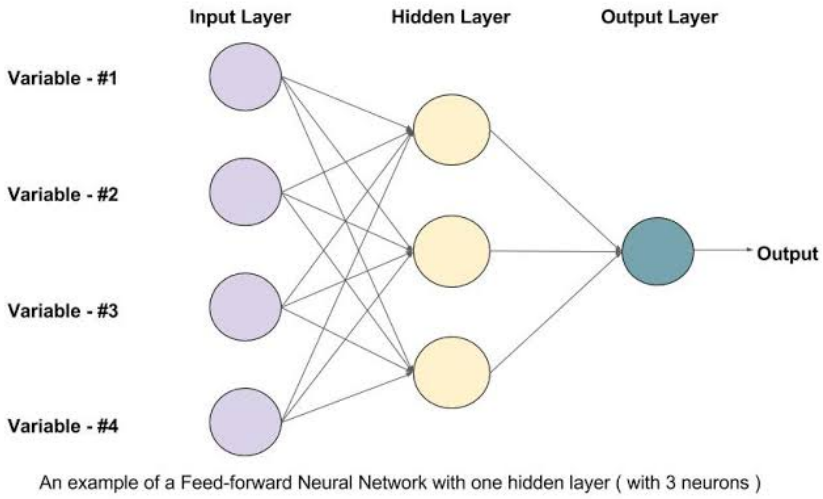


Figure 5: A small feed-forward neural network: numbers enter on the left, are mixed by weighted connections, and scores come out on the right.

Question 18 [3 marks]

A tiny vocabulary has three words $\{a, b, c\}$. The weight matrix has one row per word:

$$W = \begin{pmatrix} 1 & 2 & 0 \\ 0 & 1 & 3 \\ 2 & 0 & 1 \end{pmatrix}.$$

The current word is b, so the one-hot input is $\mathbf{x} = (0, 1, 0)$.

(a) Compute $\mathbf{z} = \mathbf{x}W$. What do you notice about the result? [2]

(b) In Figure 5, which part of the picture corresponds to the matrix W? [1]

Question 19 [4 marks]

A model produces the logits $\mathbf{z} = (0,\ \ln 3,\ \ln 6)$.

(a) Apply softmax to get the three probabilities. [2]

(b) Why does softmax use e^z and not z directly? [1]

(c) What happens to the probabilities if you add 100 to every logit? Why? [1]

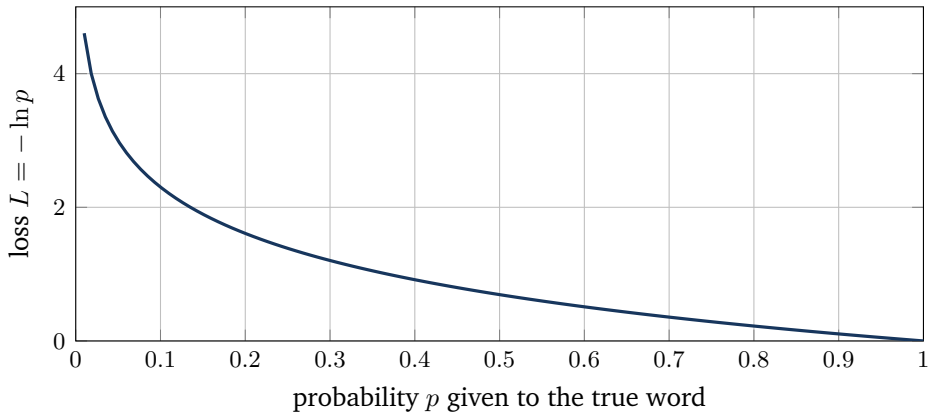


Figure 6: The loss $L = -\ln p$ as a function of the probability the model gives to the true next word.

Question 20 [4 marks]

Take the probabilities from Question 19. The *true* next word is the second one, so $p_{\text{true}} = 0.3$.

(a) Compute the loss $L = -\ln p_{\text{true}}$. [1]

(b) Use Figure 6 to estimate the loss when the model gives the true word probability 0.6, and when it gives 0.01. Then check with a calculator and describe the shape of the curve. [2]

(c) The average loss on the test sentence of Question 11 was 0.474. Compute $\exp(0.474)$ and say what this number is called. [1]

Question 21 [4 marks]

For softmax followed by this loss, the slope of the loss with respect to each logit is a beautifully simple formula:

$$\frac{\partial L}{\partial z_i} = p_i - y_i,$$

where $y_i = 1$ for the true word and 0 for all others. Use $\mathbf{p} = (0.1,\ 0.3,\ 0.6)$ and true word number 2.

(a) Write the gradient vector. [1]

(b) Do one gradient-descent step on the logits, $\mathbf{z} \leftarrow \mathbf{z} - \eta\,\nabla L$, with $\eta = 0.5$, starting from $\mathbf{z} = (0,\ 1.099,\ 1.792)$. [2]

(c) Compute the new probability of the true word to check that the step helped. [1]

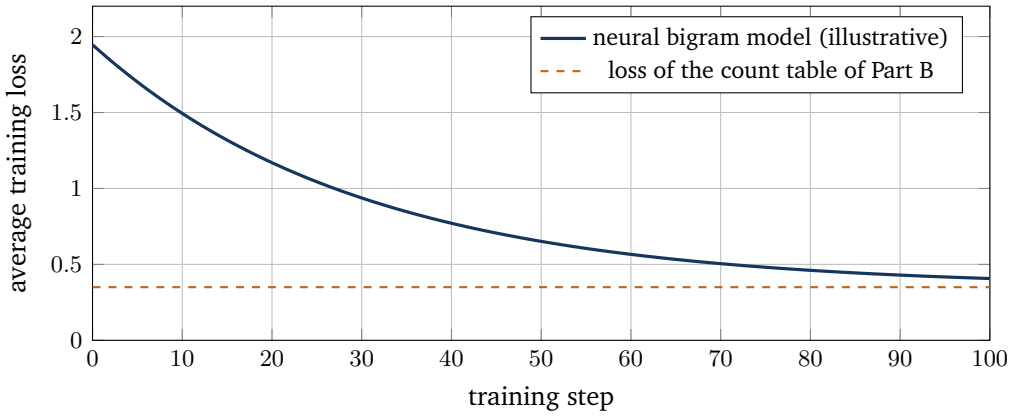


Figure 7: Training loss of a neural bigram model on the text of Part B ($V = 7$).

Question 22 [3 marks]

Figure 7 shows a neural bigram model learning the text of Part B, with the loss of the count table drawn as a dashed line.

(a) The curve starts at about 1.95. Show that this equals $\ln V$ for $V = 7$ and explain why: what is the untrained network doing? [1]

(b) The curve gets close to the dashed line but not below it. Why should the two agree? [1]

(c) Why is the neural version more useful for building bigger models? [1]

Part F: Embeddings and the neural language model

★★★ · Stretch · 10 marks

Bengio and colleagues (2003) made the network deeper. Each word gets a learned **embedding** (a short list of numbers). The embeddings of the last few words are joined together, passed through a **hidden layer**, and a final layer with softmax gives the next-word probabilities. Figure 8 shows a tiny version.

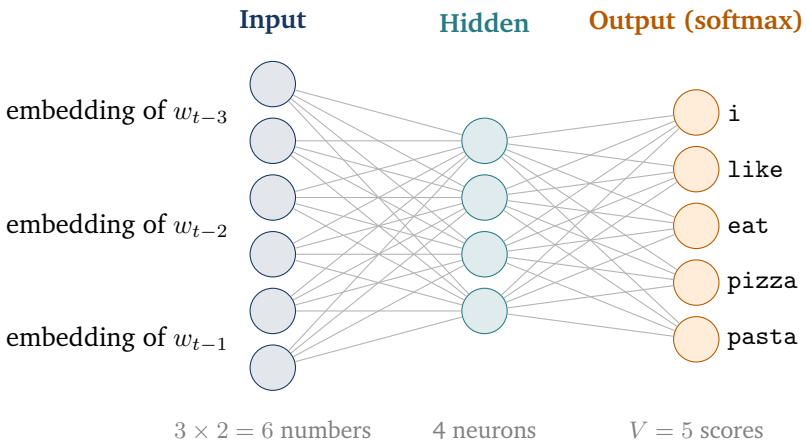


Figure 8: A tiny neural language model: vocabulary $V = 5$, embedding size $d = 2$, context of 3 words.

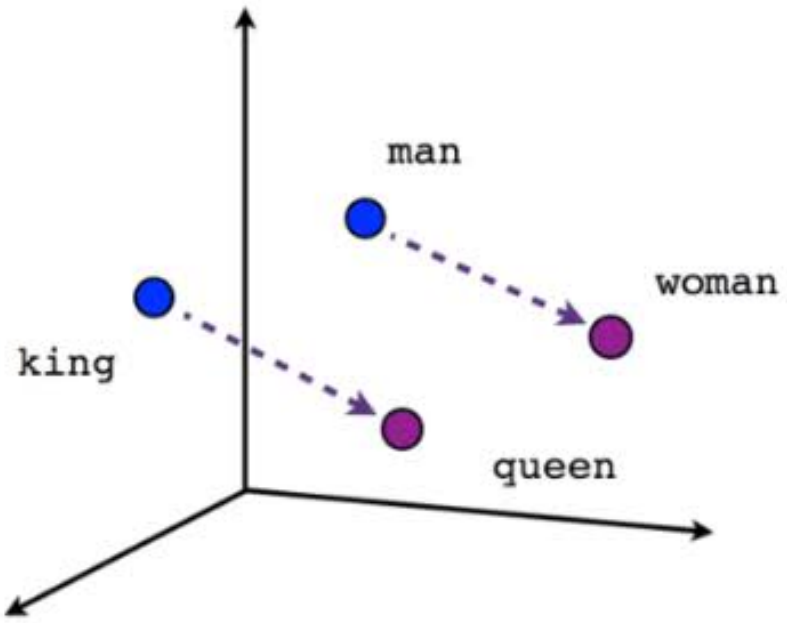


Figure 9: Learned word embeddings drawn in 2 dimensions: words with similar meanings sit close together.

Question 23 [4 marks]

A realistic model has vocabulary $V = 10{,}000$, embedding size $d = 30$, a context of 3 words and a hidden layer of 200 neurons.

(a) A count table for a 3-word context needs one entry for every context and every next word. How many entries? [1]

(b) Count the numbers the neural model must learn: the embedding table, the hidden layer (weights and 200 biases) and the output layer (weights and V biases). [2]

(c) Roughly how many times smaller is the neural model than the count table? [1]

Question 24 [3 marks]

Suppose the training text contains "the cat is walking in the bedroom" but never contains "a dog was running in a room". A count-based n-gram model would give the second sentence almost no probability. Using Figure 9, explain how embeddings let a neural model still treat it as sensible. [3]

Question 25 [3 marks]

Use Figure 8.

(a) How many weights connect the input layer to the hidden layer? [1]

(b) How many weights connect the hidden layer to the output layer? [1]

(c) Adding one bias per hidden and output neuron, and the embedding table ($V \times d$), how many numbers does this tiny model learn in total? [1]

Part G: How GPT chooses words, and speech

★★★ · Stretch · 8 marks

A GPT-style model produces logits for every possible next token. Softmax turns them into probabilities, and then the model *samples* one token. A setting called **temperature** T changes the softmax:

$$p_i = \frac{e^{z_i/T}}{\sum_j e^{z_j/T}}.$$

Question 26 [4 marks]

Three candidate next words have the logits $\mathbf{z} = (0,\ \ln 3,\ \ln 9)$. Figure 10 shows the probability of the top word (the third one) as the temperature changes.

(a) Find the probabilities for $T = 1$. [1]

(b) Find the probabilities for $T = 2$, and mark both top-word values on Figure 10. [2]

(c) Describe what happens to the curve when T is very small and when T is very large. What does a high temperature do, and what does a low temperature do? [1]

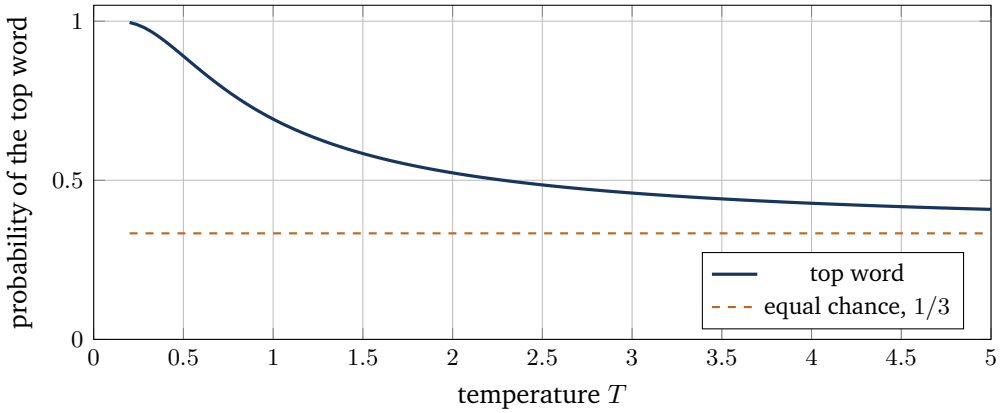


Figure 10: Probability of the most likely word as the temperature changes, for logits $(0, \ln 3, \ln 9)$.

Question 27 [2 marks]

You speak to a voice assistant and it answers out loud. Figure 11 sketches the process. Name the three main components in the correct order and say what each does. Then explain, in one sentence, why a mistake in the first component can ruin the final answer. [2]

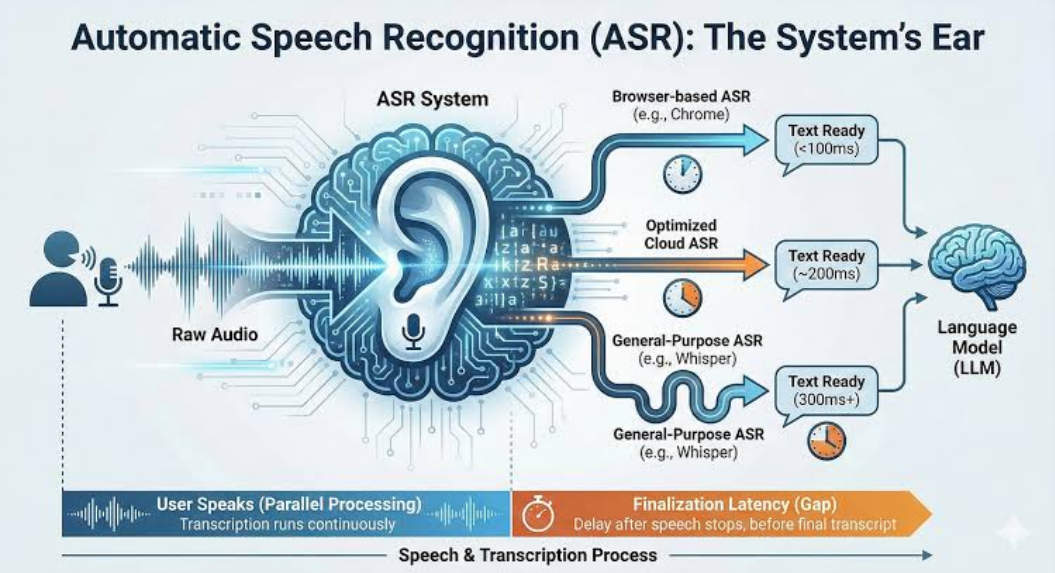


Figure 11: A voice assistant: sound goes in, sound comes out.

Question 28 [2 marks]

A user asks the same chatbot the same question twice and gets two slightly different answers. Give the most likely reason, and say how you could make the answers (almost) identical. [2]

Part H: Attention

★★★★ · Challenge · 12 marks

Attention lets each word look at the other words and decide which ones matter. For one **query** $\mathbf{q}$, a list of **keys** $\mathbf{k}_i$ and **values** $\mathbf{v}_i$:

$$\text{score}_i = \frac{\mathbf{q} \cdot \mathbf{k}_i}{\sqrt{d_k}}, \qquad \mathbf{w} = \text{softmax}(\text{scores}), \qquad \text{output} = \sum_i w_i \, \mathbf{v}_i.$$

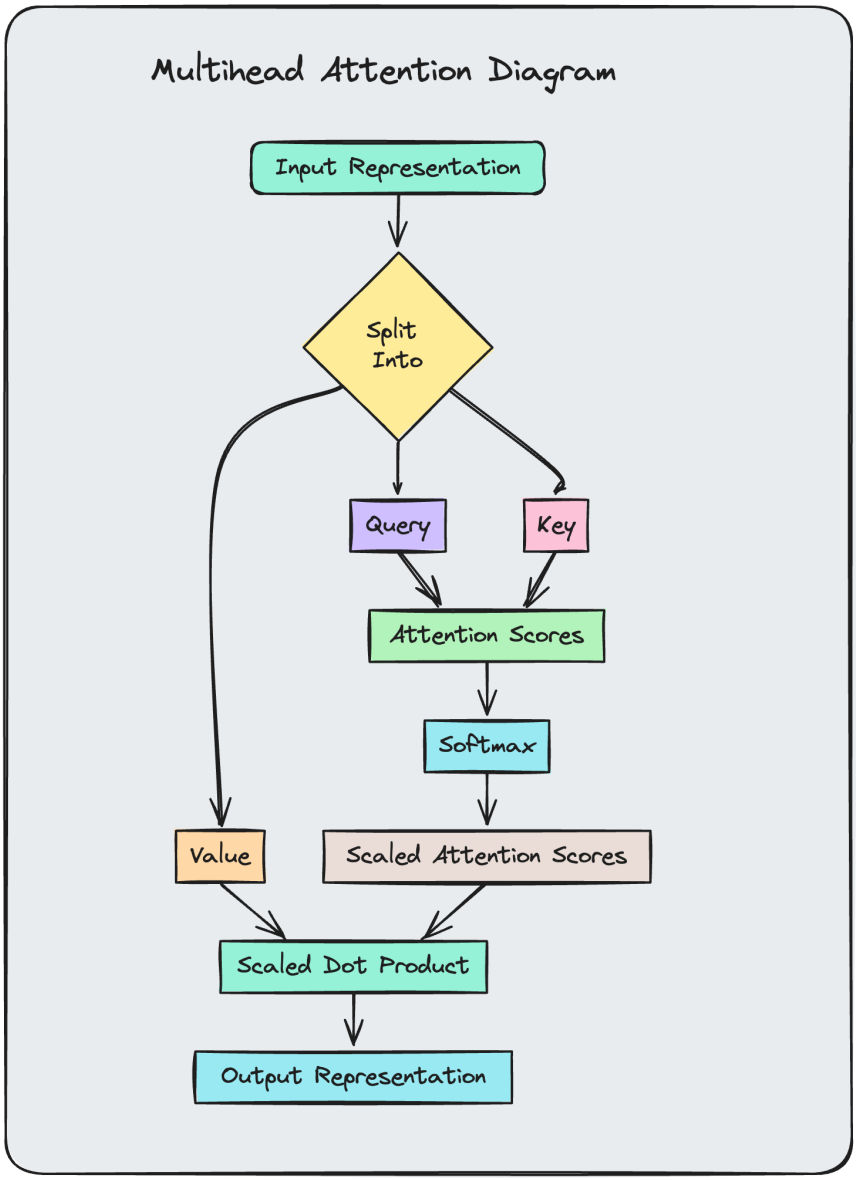


Figure 12: Attention: each word compares its query with the keys of the other words and takes a weighted mix of their values.

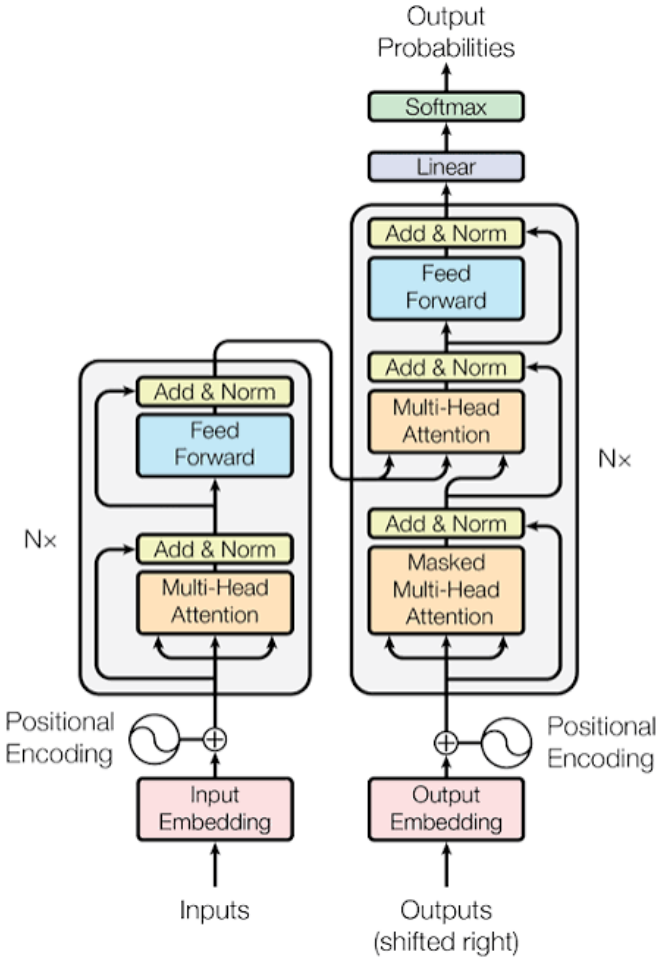


Figure 13: A GPT-style (decoder-only) transformer block, built from attention layers and small neural networks.

Question 29 [5 marks]

Let $d_k = 2$ and

$$\mathbf{q} = (1,0); \quad \mathbf{k}_1 = (1,0),\ \mathbf{k}_2 = (0,1),\ \mathbf{k}_3 = (1,1); \quad \mathbf{v}_1 = (10,0),\ \mathbf{v}_2 = (0,10),\ \mathbf{v}_3 = (5,5).$$

(a) Compute the three raw scores $\mathbf{q} \cdot \mathbf{k}_i$. [1]

(b) Divide by $\sqrt{d_k}$. [1]

(c) Apply softmax to get the attention weights. [2]

(d) Compute the output vector (2 decimal places). [1]

Question 30 [3 marks]

Why do we divide by $\sqrt{d_k}$? Think about what happens to dot products, and then to softmax, when d_k is large (for example 512). [3]

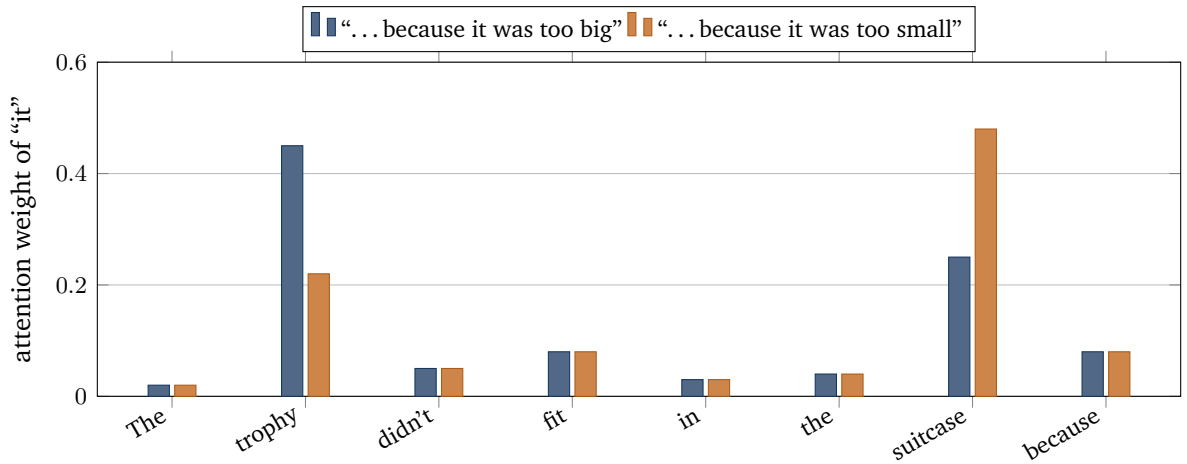


Figure 14: Illustrative attention weights from the word "it" to the earlier words of "The trophy didn't fit in the suitcase because it was too ...". Each series adds up to 1.

Question 31 [2 marks]

Figure 14 shows the attention weights of "it" in "The trophy didn't fit in the suitcase because it was too big." and in the same sentence with "small" instead of "big".

(a) Which word gets the most attention from "it" when the sentence ends with "big"? [1]

(b) Which word gets the most attention when the sentence ends with "small"? Why do the weights change when only one word of the sentence changes? [1]

Question 32 [2 marks]

A GPT-style model is trained to predict the next word, so word t may only look at words $1, \ldots, t$. Write the 4×4 mask for a 4-word sentence (1 = allowed, 0 = blocked), and explain why the model must not look at later words. [2]

Bonus challenge (up to +10 marks, not part of the 100)

1. Build it yourself. In Python or C++, write a program that

(a) reads a list of sentences, adds `<s>` and `</s>`, and counts unigrams and bigrams; [2]

(b) has a function `prob(w1, w2, k)` that returns the add-k smoothed bigram probability; [2]

(c) generates a new sentence by repeatedly sampling the next word until `</s>`; [2]

(d) computes the perplexity of a test sentence. [2]

2. Sanity check. Before training, the weights are tiny, so all logits are almost equal and every word gets probability about $1/V$. What starting loss do you expect for $V = 27$ characters? If your network starts at a loss of 25, what does that tell you? [2]

Where to read and watch more

- D. Jurafsky and J. H. Martin, *Speech and Language Processing*: the chapters on n-gram language models and on vector semantics and embeddings.
- Y. Bengio, R. Ducharme, P. Vincent and C. Jauvin, "A Neural Probabilistic Language Model", *Journal of Machine Learning Research*, 2003.
- A. Vaswani et al., "Attention Is All You Need", NeurIPS 2017.
- 3Blue1Brown, *Neural Networks* (video series), for gradient descent, softmax and the idea of attention.
- J. Alammar, "The Illustrated Transformer" (blog post).